

חני — בקרת איכות, חפיפה וסריקה

לוח QA, אישור ופסילה, שירותי חפיפה וחיפוש ברקוד

מפתחת 4 · QA Module & Services

1. פתיחה — מה לומר (30 שניות)

"שלום, אני חני. אחראית על בקרת האיכות, חפיפה וסריקה. בניתי את לוח הבקרה לבדוקות, את מערכת האישור והפסילה עם צילום חובה, את זרימת שירותי החפיפה, ואת חיפוש ההיסטוריה לפי ברקוד פאה."

2. מה עשיתי

- לוח בקרת איכות (QADashboard) — כל הפאות שממתינות לאישור במקום אחד.
- אישור עם צילום חובה — לא ניתן לאשר בלי לצלם את הפאה המוגמרת.
- פסילה עם החזרה לשלבים (RejectionModal) — בחירת שלבים + תמונת ליקוי + הערה.
- שירותי חפיפה וסירוק (ServiceOrderForm) — הזמנת שירותי סלון עם זרימה ל-QA.
- Middleware verifyQC — גישה ל-QA רק לבדוקות מורשות.
- חיפוש היסטוריה לפי ברקוד (WigHistorySearch) — מעקב מלא אחרי חיי הפאה.
- תצוגת פסילות לעובדות — "חזר מ-QA" + תמונה + הערה בכרטיס המשימה.

3. קבצים מרכזיים

תפקיד	קובץ	שכבה
לוח בקרת איכות	components/ServicesAndQA/QADashboard/QADashboard.tsx	Client
מודל פסילה	components/ServicesAndQA/RejectionModal/RejectionModal1.tsx	Client
הזמנת חפיפה/סירוק	components/ServicesAndQA/ServiceOrderForm/ServiceOrderForm.tsx	Client
חיפוש היסטוריה לפי ברקוד	components/History/WigHistorySearch.tsx	Client
getQATasks, approve, reject	Models_Service/SalonServices/serviceService.ts	Server
API — /api/services	Routers/serviceRouter.ts	Server
הרשאות בקרת איכות	Middlewares/authMiddleware.ts → verifyQC	Server
חיפוש היסטוריה	Models_Service/NewWigs/newWigService.ts → getWigHistoryByBarcode	Server
בדיקות QA approve/reject	tests/service_qa.test.ts	Tests

4. הסבר על קטעי קוד

קטע 1 — אישור עם צילום חובה (Client)

קובץ: QADashboard.tsx — handleApprove

```
const handleApprove = async (taskId) => {
  const photo = approvePhoto[taskId];
  if (!photo) {
    alert('!חובה לצלם את הפאה לפני אישור סופי');
    startCamera(taskId);
    return;
  }
  await axios.patch(`/api/services/${taskId}/approve`, {
    photoUrl: photo
  });
  alert('הפאה אושרה בהצלחה! התמונה נשמרה בהיסטוריה');
  fetchQATasks();
};
```

מה קורה כאן:

- לפני אישור — המערכת **חוסמת** אם אין תמונה ופותחת מצלמה.
- התמונה נשלחת לשרת ונשמרת כ-`afterImageUrl`.
- פאה חדשה → סטטוס "**מוכנה למסירה**". תיקון → "**מוכן**".
- התמונה נשמרת בהיסטוריה ונראית בחיפוש ברקוד.

קטע 2 — פסילה והחזרה לייצור (Client + Server)

קובץ: QADashboard.tsx — handleRejectConfirm

```
const handleRejectConfirm = async (reason, returnStages, photoUrl) => {
  await axios.patch(`/api/services/${selectedTask._id}/reject`, {
    qaNote: reason,           // סיבת הפסילה (חובה)
    returnStages,             // שלבים להחזרה (למשל: 'יצבע', 'עבודת ידי')
    photoUrl                  // תמונת הליקוי (חובה)
  });
  alert('הפאה נפסלה. התמונה וההערות נשלחו לעובדת');
};
```

מה קורה כאן:

- בפסילה חובה: **סיבה, תמונת ליקוי, ובחירת שלבים להחזרה**.
- בשרת — `rejectService` מעדכן את הפאה/תיקון: שומר `qaNote` ו-`qaRejectionPhoto`.
- לפאה חדשה — השלבים נשמרים ב-`pendingRepairStages` והפאה חוזרת לייצור.

- לתיקון — המשימות הרלוונטיות חוזרות לסטטוס ממתין.
- העובדת רואה "חזר מ-QA" + התמונה + ההערה בכרטיס המשימה.

קטע 3 — חיפוש היסטוריה לפי ברקוד (Server)

קובץ: newWigService.ts — getWigHistoryByBarcode

```
export const getWigHistoryByBarcode = async (barcode) => {
  const wig = await NewWig.findOne({
    orderCode: { $regex: new RegExp(`^${barcode.trim()}$`, 'i') }
  }).populate('customer');

  if (!wig) throw new AppError('404', 'לא נמצאה פאה עם קוד זה');

  const repairs = await Repair.find({ wigCode: wig.orderCode });
  const serviceHistory = await Service.find({ newWigReference: wig._id });

  return {
    wigDetails: wig,
    repairHistory: repairs,
    productionHistory: wig.history,
    serviceHistory: serviceHistory
  };
};
```

מה קורה כאן:

- חיפוש לפי orderCode — לא רגיש לאותיות גדולות/קטנות.
- מחזיר היסטוריה מלאה: פרטי פאה, ייצור, תיקונים, שירותים ו-QA.
- ב-Client (WigHistorySearch) — מזינים קוד בדף /history ורואים הכל.
- סורק ברקוד פיזי עובד כהקלדה אוטומטית בשדה הטקסט.

קטע 4 — חפיפה בזרימת הייצור

קובץ: workflowHelper.ts

```
const STAGE_WORKFLOW = {
  'עבודת יד': { next: 'חפיפה', specialty: 'חפיפה' },
  'חפיפה': { next: 'בקרה', specialty: 'בקרת איכות' },
  'בקרה': { next: 'הושלם', specialty: null }
};
```

מה קורה כאן:

- **חפיפה** = שלב שטיפה ועיצוב לפני מסירה ללקוחה (לא "חפיפת משמרות").

- בפאה חדשה: שלב 5 מתוך 6, לפני בקרה.
- בתיקונים: נוצרת אוטומטית על ידי מרימי בסיום התיקונים.
- גם שירותי סלון (חפיפה וסירוק) עוברים דרך אותו לוח QA.

5. תסריט הדגמה

1 התחברי כ- רחלי / password123 (בקרת איכות) → מגיעים ל- qa/ .

2 הציגי רשימת פאות ממתיונות — פאה חדשה (תג "פאה חדשה") + תיקון (תג "תיקון").

3 לחצי על פאה — ראי תמונת "לפני" מהייצור/תיקון.

4 לחצי "צלם לאישור" — צלמי את הפאה המוגמרת.

5 לחצי "אשר" — הפאה עוברת ל"מוכנה למסירה".

6 (אופציונלי) הדגימי פסילה — בחרי שלבים להחזרה, צלמי ליקוי, כתבי הערה.

7 עברי ל- history/ — הזיני קוד פאה וראי היסטוריה מלאה כולל תמונות QA.

8 עברי ל- service/new/ — הדגימי הזמנת שירות חפיפה וסירוק.

6. טכנולוגיות

Mongoose

Axios

MediaDevices API (מצלמה)

TypeScript

React

Jest

MongoDB Regex Search

Express Middleware (verifyQC)

7. שאלות ותשובות למורה

ש: למה חובה צילום לפני אישור?

הצילום (afterImageUrl) הוא הוכחה ויזואלית שהפאה נבדקה ואושרה. הוא נשמר ב-DB ומופיע בהיסטוריה — כך אפשר לעקוב אחרי מצב הפאה בכל שלב. בלי תמונה — כפתור האישור חסום (client validation). זה מבטיח שכל פאה שיוצאת מהסלון עברה בדיקה פיזית.

ש: מה קורה בפסילה?

הבודקת בוחרת שלבים להחזרה (למשל: צבע, עבודת יד), מצלמת את הליקוי, וכותבת הערה. בשרת: לפאה חדשה — השלבים נשמרים ב-pendingRepairStages והפאה חוזרת לייצור. לתיקון — המשימות הרלוונטיות חוזרות ל-ממתין. העובדת רואה בכרטיס: "חזר מ-QA" + תמונת הליקוי + ההערה.

ש: איך עובד חיפוש ברקוד?

בדף /history, מזינים את קוד הפאה (orderId). השרת מחפש ב-MongoDB עם Regex (לא רגיש לאותיות). מחזיר: פרטי לקוחה, היסטוריית ייצור (מי עבדה על מה), תיקונים, שירותים, תמונות QA. סורק ברקוד פיזי ממלא את השדה אוטומטית (כמו הקלדה).

ש: מה זה verifyQC?

Middleware בשרת שמגביל גישה לנתיבי (/api/services/qa-tasks, approve, reject). QA. מאפשר גישה ל: תפקיד QC או Inspector, מנהלת, או עובדת עם התמחות "בקרת איכות" / "בקרה". עובדת רגילה תקבל 403.

ש: מה ההבדל בין Production QA ל-Repair QA?

שניהם מופיעים באותו לוח (/qa), אבל: Production QA (origin: 'NewWig') — נוצר אוטומטית כשפאה חדשה מסיימת ייצור (מרים). Repair QA (origin: 'Repair') — נוצר אוטומטית כשתיקון מגיע לבקרה (מרימי). בלוח QA יש תג שמבדיל ביניהם. הלוגיקה של אישור/פסילה זהה.

ש: מה זה חפיפה במערכת?

חפיפה = שלב שטיפה, ייבוש ועיצוב הפאה לפני מסירה ללקוחה. בפאה חדשה — זה שלב 5 בזרימת הייצור. בתיקונים — נוצר אוטומטית בסיום. גם יש שירותי סלון נפרדים: "חפיפה בלבד" או "חפיפה וסירוק" דרך `ServiceOrderForm`, שגם הם עוברים דרך QA לפני מסירה.

ש: איך QA מתחבר לשאר המודולים?

מרים (פאה חדשה) — בסיום ייצור, `newWigService` יוצר `Service` עם `QA` status: 'מרימי (תיקונים) — בהגעה לבקרה, `repairService` יוצר `QA` Repair חני (QA) — רואה הכל ב-QADashboard, מאשרת/פוסלת, והתוצאה חוזרת למודול המקור. אילה (דאשבורד) — רואה את הסטטוס המעודכן בטבלת הסקירה.

ש: אילו בדיקות כתבת?

`service_qa.test.ts` — בדיקות: שליפת משימות QA, אישור עם תמונה, פסילה עם החזרה לשלבים, בדיקה שהערות ותמונות נשמרות נכון. `e2e_workflow.test.ts` — זרימה מלאה כולל פסילת QA והחזרה לייצור.